5-5 · 파티셔닝과 셔플링 최적화 · 이론 1H

파티셔닝과 셔플링 원리

파티션은 병렬의 단위, 셔플은 가장 비싼 비용

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 파티션이 태스크 병렬성의 단위임을 설명한다

② 좁은 변환과 넓은 변환을 구분하고 셔플이 생기는 지점을 짚는다

③ 실행 계획에서 Exchange 를 세어 셔플 여부를 판정한다

④ repartition 과 coalesce 의 차이를 표로 말한다

다음 1.5H 실습에서 파티션 수를 바꿔 가며 실행 시간과 셔플 지표를 직접 잽니다

파티션 = 일감의 조각 — 조각 하나가 태스크 하나

데이터	파티션	태스크	익스큐터 코어
1천만 행	일감의 조각	조각 하나에 태스크 하나	코어 하나가 태스크 하나

병렬의 폭 = 파티션 수와 코어 수 중 작은 쪽

파티션이 4개면 코어가 아무리 많아도 동시에 4개까지만 일한다 — 반대로 너무 잘게 나누면 조각마다 준비 비용

파티션 수가 맞지 않으면 생기는 문제

1. **너무 적으면**
   코어가 놀고, 한 조각이 커서 메모리 부족
2. **너무 많으면**
   조각마다 준비 비용 · 작은 파일이 쏟아짐
3. **크기가 들쭉날쭉하면**
   한 태스크만 오래 걸려 전체가 기다림 — UI 의 Task Max/Median 차이로 보임

흔히 쓰는 눈대중

100~200MB

코어의 2~3배

> **절대 규칙은 아니다 — 5-1 에서 본 Spark UI 의 Task 시간 분포를 보고 조정**

2. 셔플

좁은 변환은 제자리, 넓은 변환은 이동

좁은 변환 — 제자리

각 파티션 안에서 끝난다 — 이동 없음

- filter · withColumn · select
- 네트워크를 건너지 않는다
- Stage 가 나뉘지 않는다

VS

넓은 변환 — 이동

같은 키끼리 모이려 파티션을 넘나든다

- groupBy · join · distinct · orderBy
- repartition 도 넓은 변환
- 넓은 변환이 Stage 의 경계

셔플 = 조별 재편성 — 가장 비싼 비용

Shuffle Write	네트워크	Shuffle Read
각 파티션이 디스크에 쓰고	다른 익스큐터로 보내고	받는 쪽이 다시 읽는다

메모리 안에서 끝나는 연산보다 수십 배 비싸다

학급 전체가 「성씨별로 모여!」 — 모두 일어나 교실을 가로질러 이동하는 것과 같다

실행 계획에서 셔플 찾기 — Exchange 를 센다

1. **explain() 으로 물리 계획을 본다**
   Exchange hashpartitioning 이 곧 셔플
2. **Exchange 가 몇 개인지 센다**
   튜닝의 첫걸음 — 데이터가 몇 번 이동하는가
3. **HashAggregate 가 위아래 두 번**
   셔플 전에 각 파티션에서 미리 집계해 보낼 양을 줄인 것 — 부분 집계

계획에서 찾을 글자

Exchange

> **그래서 count·sum 은 셔플이 있어도 비교적 저렴하고, distinct 나 collect_list 는 비싸다**

repartition(n) 과 coalesce(n)

	repartition(n)	coalesce(n)
하는 일	전체를 셔플해 고르게 다시 나눔	이웃 파티션을 합치기만
셔플	있음 (비쌈)	없음 (저렴)
수를 늘릴 수	있음	없음 — 줄이기만
결과 분포	균등	원래 조각이 고르지 않으면 치우침
언제	스큐를 풀 때 · 병렬을 늘릴 때	저장 전 파일 수를 줄일 때

쓰기 파일 수 = 저장 직전 파티션 수 — 작은 마트는 coalesce(1), 아주 큰 데이터는 한 태스크가 떠안으니 크기를 보고

5. 줄이는 방법

셔플을 줄이는 여섯 가지

방법	무엇	언제
필터를 먼저	셔플 전에 행을 줄인다	거의 항상
필요한 열만 select	옮길 양을 줄인다	거의 항상
브로드캐스트 조인	작은 쪽을 복사해 셔플 회피	한쪽이 작을 때
shuffle.partitions	기본 200이 안 맞을 때 조정	작으면 줄이고 크면 늘림
파티션 컬럼으로 저장	읽을 때 셔플 없이 걸러짐	자주 거는 조건이 있을 때
AQE 켜기	실행 중 Spark 가 알아서 조정	Spark 3.x 기본 활성

Spark 3 부터 AQE 가 기본 — 실행 중 셔플 파티션을 알아서 합쳐 주므로 shuffle.partitions 를 바꿔도 차이가 작다. 그래도 원리를 알아야 UI 를 읽는다

정리

이 1시간에서 꼭 가져갈 것

1. 파티션 하나가 태스크 하나 — 너무 적어도 많아도 손해다
2. 셔플은 디스크·네트워크를 거치는 가장 비싼 연산이다
3. 좁은 변환은 제자리, 넓은 변환은 이동 — 넓은 변환이 Stage 를 가른다
4. 실행 계획에서 Exchange 를 세는 것이 튜닝의 첫걸음이다
5. repartition 은 셔플 있음·늘리기 가능, coalesce 는 셔플 없음·줄이기만
6. 저장 직전 파티션 수가 출력 파일 수다

5-5 · 이론 1H 마무리

다음은 repartition/coalesce 실습 (1.5H)

파티션 수를 바꿔 가며 분포와 Spark UI 의 셔플 지표를 직접 재고, 스큐를 일부러 만들어 봅니다

- ✓ **getNumPartitions() 로 파티션 수 변화 확인**
- ✓ **Jobs·Stages 탭에서 Shuffle Read/Write 읽기**
- ✓ **AQE 를 끄고 shuffle.partitions 200·8 비교**
- ✓ **스큐를 만들어 Task Max/Median 차이 보기**

준비할 것

실습점검으로 4040 열기 · bash lake-pyspark.sh